Introduction to Programming and Data Structures

Analysis of Algorithms

Malay Bhattacharyya

Associate Professor

MIU, CAIML, TIH
Indian Statistical Institute, Kolkata

November, 2023

1 Basics

2 Asymptotic notation

3 Problems

Order of growth

Time Complexity is the study of the efficiency of algorithms. It tells us how much time is taken by an algorithm to process a given input.

big O stands for 'order of'. Big O in mathematics stands for all those complexities our program runs in, but in industry, we are asked the minimum of them.

The order of growth of the running time of an algorithm may provide a simple (yet quantitative) characterization of the algorithm's efficiency.

This may allow us to compare the relative performance of alternative algorithms.

Drop the constants:
Anything you might think is O(kn) (where k is a constant) is O(n) as well. This is considered a better representation of the time complexity since the k term would not affect the complexity much for a higher value of n.

Drop the non-dominant terms: :
Anything you represent as O(n2+n) can be written as O(n2).

Consider all variables which are provided as input:
O (mn) and O (mnq) might exist for some cases.

The motivation

Suppose we want to quantitatively analyze the running time of an algorithm.

- Can we consider the running time as a function of the input size?
- Can we study how the function behaves if we let it run when the input size grows toward the infinity?
- Can we put bounds on these functions?
- How can we say that the function is arbitrarily close to the bound?

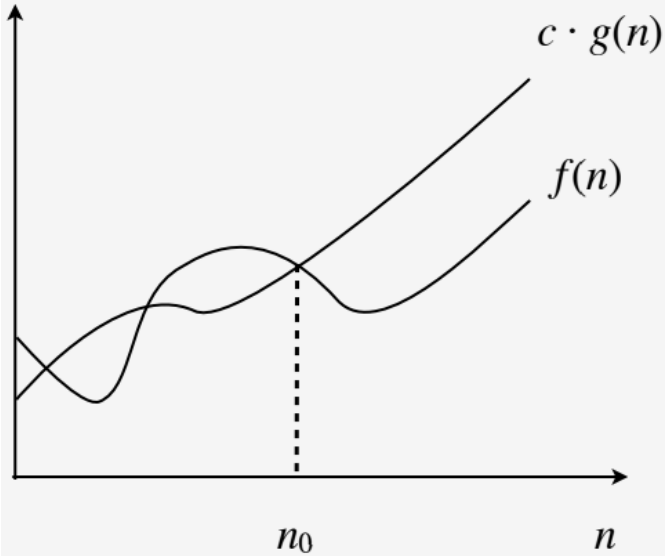
Note: The term `asymptotic` refers to approaching a value or curve arbitrarily closely.

To calculate the time complexity of an algorithm, you can follow these steps
1:Identify the critical operations that dominate the algorithm's runtime.
2. Count the number of times these critical operations are executed based on the input size.
3. Express the number of critical operations as a function of the input size.

O Notation

Big oh notation is used to describe an asymptotic upper bound.

The function $f(n)$ is $O(g(n))$ if there exist constants $c > 0$ and $n_0 \geq 0$ such that $0 \leq f(n) \leq c * g(n)$ for all $n \geq n_0$.



For example, consider $f(n) = 12n^2 + 5n + 10$.

- $f(n)$ is $O(n^2)$ (non-trivial)
- $f(n)$ is $O(n^2 \log n)$, $O(n^3)$, $O(n^3 \log n)$, $O(n^4)$, ... (trivial)
- $f(n)$ is neither $O(n)$ nor $O(n \log n)$.

Note: $f(n)$ and $g(n)$ are real-valued functions.

O Notation (contd.)

Remember that $O(g(n))$ is a set of functions, but we do often write $f(n) = O(g(n))$ instead of $f(n) \in O(g(n))$. This is a notational abuse and has to be handled carefully.

For example, consider $f_1(n) = 2n^2$ and $f_2(n) = 5n^3$.

- One can write $f_1(n) = O(n^3)$ and $f_2(n) = O(n^3)$
- However, $f_1(n) = f_2(n)$ can not be concluded

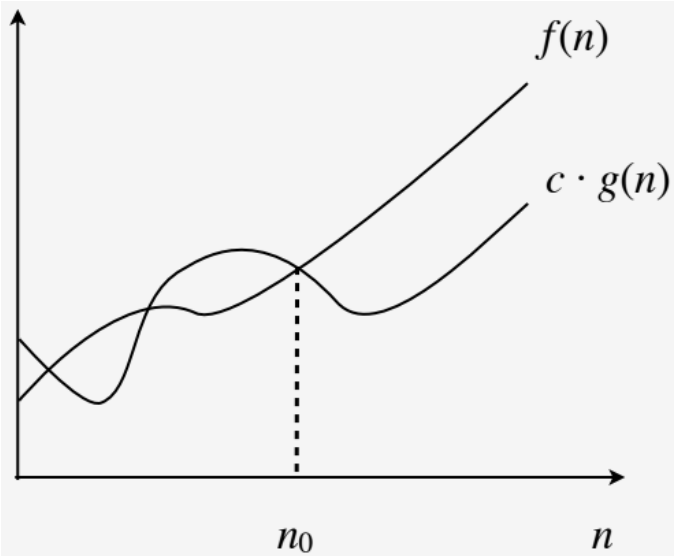
O Notation (contd.)

- $f(n)$ is $O(f(n))$ (reflexivity)
- If $f(n)$ is $O(g(n))$ and $c > 0$, then $c * f(n)$ is $O(g(n))$ (constant bounding)
- If $f_1(n)$ is $O(g_1(n))$ and $f_2(n)$ is $O(g_2(n))$, then $f_1(n) * f_2(n)$ is $O(g_1(n) * g_2(n))$. (product bounding)
- If $f_1(n)$ is $O(g_1(n))$ and $f_2(n)$ is $O(g_2(n))$, then $f_1(n) + f_2(n)$ is $O(\max g_1(n), g_2(n))$. (summation bounding)
- If $f(n)$ is $O(g(n))$ and $g(n)$ is $O(h(n))$, then $f(n)$ is $O(h(n)$. (transitivity)

Ω Notation

notation provides an asymptotic lower bound.

The function $f(n)$ is $\Omega(g(n))$ if there exist constants $c > 0$ and $n_0 \geq 0$ such that $f(n) \geq c * g(n) \geq 0$ for all $n \geq n_0$.



For example, consider $f(n) = 12n^2 + 5n + 10$.

- $f(n)$ is $\Omega(n^2)$ (non-trivial)
- $f(n)$ is $\Omega(n^0 = 1)$, $\Omega(n)$ (trivial)
- $f(n)$ is not $\Omega(n^3)$.

Note: $f(n)$ and $g(n)$ are real-valued functions.

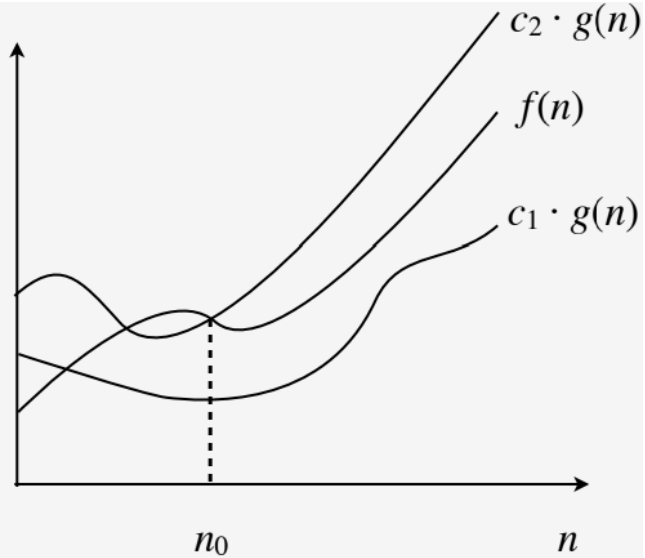
Ω Notation (contd.)

- $f(n)$ is $\Omega(f(n))$ (reflexivity)
- If $f(n)$ is $\Omega(g(n))$ and $c > 0$, then $c * f(n)$ is $\Omega(g(n))$ (constant bounding)
- If $f_1(n)$ is $\Omega(g_1(n))$ and $f_2(n)$ is $\Omega(g_2(n))$, then $f_1(n) * f_2(n)$ is $\Omega(g_1(n) * g_2(n))$. (product bounding)
- If $f_1(n)$ is $\Omega(g_1(n))$ and $f_2(n)$ is $\Omega(g_2(n))$, then $f_1(n) + f_2(n)$ is $\Omega(\min g_1(n), g_2(n))$. (summation bounding)
- If $f(n)$ is $\Omega(g(n))$ and $g(n)$ is $\Omega(h(n))$, then $f(n)$ is $\Omega(h(n)$. (transitivity)

Θ Notation

Let f(n) define the running time of an algorithm.
F(n) is said to be (g(n)) if f(n) is O (g(n)) and f(x) is (g(n)) both.

The function $f(n)$ is $\Theta(g(n))$ if there exist constants $c_1, c_2 > 0$ and $n_0 \geq 0$ such that
$0 \leq c_1 * g(n) \leq f(n) \leq c_2 * g(n)$ for all $n \geq n_0$.



For example, consider
$f(n) = 12n^2 + 5n + 10$.

- $f(n)$ is $\Theta(n^2)$
- $f(n)$ is neither $\Theta(n \log n)$ nor $O(n^2 \log n)$.

Note: $f(n)$ and $g(n)$ are real-valued functions.

Problems

BINARY SEARCH TIME COMPLEXITY : log(n) refers to how many times I need to divide n units until they can no longer be divided (into halves).

1. Suppose $f(n) = n^2 + 139n \log n + 500$. What is the $O()$ of the function? O(n^2)
2. Suppose $f(n) = 10$. What is the $O()$ of the function? O(1)

3) Show that f (n) is (g(n)) iff f (n) is both O(g(n)) and (g(n)).
ans : d * g(n) <= f(n) <= c * g(n)

O(n) (n)

4) Let f (n) = a0 + a1n + . . . + ad nd with ad > 0. Then prove that f (n) is (nd).
ans : Since ad * n^d <= c * n^d for sufficiently large n, it follows that:
f(n) = a0 + a1n + ... + ad * n^d <= (a0 + a1n + ... + ad * n^d) <= (a0 + a1n + ... + c * n^d) = c' * n^d
For some c' > 0, f(n) is O(n^d).
Since c * n^d <= ad * n^d for sufficiently large n, it follows that:
f(n) = a0 + a1n + ... + ad * n^d >= (a0 + a1n + ... + ad * n^d) >= (a0 + a1n + ... + c * n^d) = c' * n^d
For some c' > 0, Therefore, f(n) is (n^d).